



Birmingham City University

Faculty of Computing, Engineering and the Built Environment

CMP6207 Modern Data Stores

Smart Tank Automation:

**A Distributed, Fault-Tolerant NoSQL Telemetry
Platform for IoThings**

Student Name:	Shekhar Lamichhane Magar
Student ID:	23189647
Programme:	BSc (Hons) Computer and Data Science
Module:	CMP6207 Modern Data Stores
Assessment:	Coursework Report
Submission:	October 2026
Main Narrative:	~4,000 words
Academic Year:	2025–26

Contents

1	Introduction	1
2	Principal NoSQL Types and Theoretical Basis	2
2.1	Key–Value Stores	2
2.2	Document Stores	3
2.3	Wide-Column Stores	3
2.4	Graph Stores	4
2.5	Theoretical Trade-offs: Storage Engines, CAP and PACELC	4
3	Critical Comparison: Relational and Document Databases	5
3.1	Schema Evolution and Normalisation	5
3.2	Query Performance and Indexing Efficiency	6
3.3	ACID Guarantees and Idempotent Messaging	7
3.4	Distributed Scalability, Partitioning Schemes and Horizontal Bounds	8
4	IoT Things Database Design and Implementation	9
4.1	Architecture and Local Deployment	9
4.2	Database Schema, Validation and Collections	10
4.3	Replication, Quorum and Empirical Failover	11
4.4	Disaster Recovery: Backup and Restore	14
5	API Implementation and Dashboard Evidence	15
5.1	Water Intelligence: Analytics, Leak Heuristics and Depletion Prediction	20
6	Conclusion and Future Enhancements	20
6.1	Summary of Findings	20
6.2	Critical Evaluation and Limitations	21
6.3	Future Enhancements and Roadmap	22
6.4	Conclusion	23
	References	24
	Appendices	27
	Appendix A: Dataset Provenance and Sample BSON Document	27
	Appendix B: Local Replica Set Deployment Commands	29
	Appendix C: Verification and Test Suite Status	30

List of Figures

1	Schema evolution: heterogeneous firmware payloads coexisting in one collection. . .	6
2	Indexes on <code>sensor_activations</code> inspected in MongoDB Compass.	7
3	Compound indexes on the <code>alerts</code> collection inspected in MongoDB Compass. . .	8
4	System architecture: write path (solid) and read path (dashed).	9
5	Process table confirming three isolated local MongoDB daemon processes.	10
6	Node.js ingestion service: insert, dead-letter routing and duplicate handling. . . .	10
7	Replica set status: one primary and two secondaries, all healthy.	12
8	Document counts across replica set members (within one document, as ingestion continued) and measured replication lag.	12
9	Graceful step-down failover probe: 19 of 19 writes acknowledged, longest pause 0.62 s, no acknowledged write lost.	13
10	Abrupt primary termination (<code>kill -9</code>): longest write pause 11.48 s; 67 of 67 acknowledged writes present afterwards; 12 writes during the outage not acknowledged.	14
11	Quorum loss with two of three members stopped: the majority write is not acknowledged (write-concern timeout).	14
12	Backup and restore with full document-count parity.	15
13	Swagger UI interactive API documentation.	16
14	Successful paginated query: <code>GET /api/telemetry/history</code>	17
15	API security: enforcement of the <code>X-API-Key</code> header.	17
16	CRUD operations demonstrated through the API.	18
17	Live React dashboard with water-intelligence analytics.	18
18	Tank gauge, sensor states and recent alerts in the React interface.	19
19	React cluster diagnostics and failover monitoring view.	19
20	MongoDB Compass document explorer showing the BSON data structure.	28

List of Tables

1	Principal NoSQL families, their benefits and drawbacks, and fit for smart tank telemetry.	3
2	Relational and document databases compared (generic, not product-specific). . .	5
3	Query plan benchmark on 30,985 records	7
4	Collections in database <code>smart_water</code>	11
5	CRUD operations exposed by the API.	16
6	Objectives, outcomes and supporting evidence.	21

7	Phased enhancement roadmap and gate criteria (effort is indicative).	22
8	Datasets used to test the implementation.	27

1 Introduction

IoThings Home Automation Solutions is a UK-based enterprise that designs, installs and manages connected smart-home sensors and automation actuators. Its enterprise resource planning (ERP), customer relationship management (CRM), inventory and utility billing systems run on relational database management systems. Relational engines excel at structured business transactions governed by ACID invariants, but they face operational friction when asked to absorb a continuous, high-velocity, append-heavy stream of device telemetry whose schema changes whenever microcontrollers receive over-the-air firmware updates.

This report evaluates MongoDB, a distributed document-oriented NoSQL database, as a dedicated telemetry storage and automation engine for IoThings. The operational prototype models a rooftop water tank subsystem. A hardware hub, `HOME_HUB_01`, samples an ultrasonic depth sensor and two safety float switches, and it controls an inlet refill valve and a pressurised booster pump. The prototype represents a standard 2,000-litre household reservoir that is 200 cm high.

In brief for IoThings management. The prototype gives three reasons to adopt MongoDB for the telemetry tier. First, devices can change their payload format after a firmware update without a database migration (Figure 1). Second, the most common query, the latest readings for one device, took 1 ms with an index instead of 105 ms by scanning 30,985 records (Table 3). Third, a three-member replica set kept every acknowledged write through a primary failure, at the cost of a pause of about 11 s (Section 4.3). IoThings should keep its relational systems for billing, CRM and inventory, and add MongoDB beside them. The prototype is a validated design, not a production system, and Section 6 sets out what remains to be done.

Figure 4 shows the complete pipeline. A physics-based simulator publishes JSON telemetry to an Eclipse Mosquitto broker at MQTT quality of service (QoS) 1, modelled on an ESP32-class microcontroller. A Node.js ingestion engine consumes the stream, validates each payload against structural schemas, evaluates closed-loop control rules and writes timestamped documents to a three-member MongoDB replica set (`rs0`). An Express REST API, documented with Swagger UI, exposes the data to a React dashboard that refreshes in near real time. Aggregation pipelines inside MongoDB execute consumption analytics in place, while leak and depletion heuristics turn the stored telemetry history into automated early warnings.

The report pursues four objectives:

1. Analyse the architectural foundations, storage engines and partitioning schemes of the four principal NoSQL families.
2. Critically compare relational and document paradigms for schema evolution, transaction semantics, indexing and distributed scalability.
3. Document the design and distributed administration of the MongoDB smart tank cluster,

demonstrating failover resilience, indexing benefit and disaster recovery with measured evidence.

4. Provide an executive roadmap of phased enhancements, each with a measurable gate, for production deployment.

The central operational finding is that a three-member replica set delivers automatic failover without losing an acknowledged write. A graceful step-down paused writes for 0.62 s, and an abrupt primary kill paused them for 11.48 s, during which 12 probe writes were not acknowledged; every acknowledged write survived. The platform therefore offers high availability, but it does not claim zero downtime.

Section 2 reviews the four NoSQL families and the theory that separates them. Section 3 compares document and relational designs. Section 4 describes the implementation and the distributed experiments, and Section 5 covers the API, dashboard and water-intelligence features. Section 6 summarises the findings, evaluates their limits and proposes the roadmap. All measurements were taken on a single development machine using scripts in the project repository, with raw outputs kept under **evidence/**. They indicate behaviour rather than production capacity, and Section 6.2 discusses this limit.

2 Principal NoSQL Types and Theoretical Basis

NoSQL (“Not Only SQL”) systems arose in response to workloads that overwhelmed monolithic relational engines: unconstrained write velocity, heterogeneous schemas, and the need to scale horizontally across commodity nodes (Cattell, 2011; Sadalage and Fowler, 2012). The name matters: NoSQL extends the relational toolbox rather than replacing it, and several systems, such as Cassandra with its SQL-like CQL, deliberately keep a familiar query style. NoSQL systems are usually grouped into four families according to data model, storage engine and partitioning strategy (Table 1). Each family is assessed against the demands of IoThings telemetry: a steady write rate, flexible and evolving payloads, time-ordered queries per device, aggregate analytics and high availability.

2.1 Key–Value Stores

Key–value databases, such as Amazon Dynamo and Redis, treat records as opaque byte payloads indexed strictly by a unique primary key (DeCandia et al., 2007). Consistent hashing partitions keys across a logical ring, minimizing remapping during node additions. While primary-key lookups achieve $O(1)$ constant time, the engine cannot inspect nested fields. Range or temporal queries (e.g., computing average night-time consumption) require scanning the entire keyspace or maintaining application-managed secondary indexes. In IoThings, key–value stores excel at caching instantaneous device state, but cannot serve as the primary telemetry historical engine.

Table 1: Principal NoSQL families, their benefits and drawbacks, and fit for smart tank telemetry.

Family	Data Representation	Main Benefit	Main Drawback	Fit for IoT Things Telemetry
Key-Value (Redis, Dynamo)	Opaque byte arrays behind unique keys	Constant-time lookup by key; simple scale-out by hashing keys	Cannot query inside values; range queries need full scans	Caching the latest device state; poor for analytical time-series queries.
Document (MongoDB)	Semi-structured hierarchical JSON/BSON documents	Flexible schema, compound indexing, server-side aggregation, built-in replication and sharding	Many-to-many joins are awkward; 16 MB document cap	Ideal: native JSON mapping and in-database analytics.
Wide-Column (Cassandra, Bigtable)	Sparse multidimensional sorted maps (row, column, time)	Very high write throughput; masterless scale-out	Query-first table design; limited ad-hoc queries	Hyperscale ingestion; requires rigid design and excessive overhead at this scale.
Graph (Neo4j)	Nodes, directed edges and key-value properties	Fast multi-hop relationship traversal via direct pointers	Poor fit for flat, high-rate append streams	Unsuitable for telemetry streams; valuable for topological asset tracking.

2.2 Document Stores

Document databases store polymorphic, hierarchical data in BSON format, extending JSON with 64-bit integers, dates and ObjectIds. Related telemetry facts—ultrasonic water depth, safety float switches, and actuator states—are embedded within a single aggregate document and committed atomically. Server-side aggregation pipelines execute analytics in place, avoiding expensive network data transfer. Schema validation (`$jsonSchema`) enforces structural rules on mandatory keys while permitting dynamic optional fields (MongoDB, Inc., 2026d). Because documents are capped at 16 MB, readings are modelled as discrete chronological events rather than unbounded embedded arrays.

2.3 Wide-Column Stores

Wide-column stores, such as Google Bigtable (Chang et al., 2008) and Apache Cassandra (Lakshman and Malik, 2010), organise data as multi-dimensional sorted maps indexed by row key, column family, column qualifier and timestamp. Cassandra uses a masterless peer-to-peer ring that applies Dynamo-style consistent hashing and a gossip protocol, so no single node is a point of failure (Lakshman and Malik, 2010; DeCandia et al., 2007).

At the storage layer, Cassandra implements a Log-Structured Merge-tree (LSM-tree) architecture (O’Neil et al., 1996). Ingestion appends sequentially to an on-disk commit log and buffers in an in-memory Memtable, giving amortised constant-time writes with no in-place disk

updates. A full Memtable is flushed to disk as an immutable Sorted String Table (SSTable), and in-memory Bloom filters let reads skip SSTables that cannot hold a key. Background compaction merges SSTables and purges tombstones, which introduces periodic I/O amplification and read latency jitter. Consistency is tunable through quorum arithmetic ($R + W > N$, where R is read consistency, W is write consistency, and N is replication factor) (DeCandia et al., 2007), so developers can choose per request between strong consistency (LOCAL_QUORUM) and minimal latency (ONE). For IoThings, wide-column stores deliver outstanding ingestion throughput for massive fleets, but querying is strictly constrained to predetermined partition and clustering keys. Ad-hoc time-window aggregations, moving averages and leak heuristics require auxiliary materialized views or integrated Apache Spark analytics, imposing excessive architectural overhead on regional IoT deployments.

2.4 Graph Stores

Graph databases, such as Neo4j, represent entities as nodes and typed relationships, both storing arbitrary key-value attributes (Robinson et al., 2015). The architectural foundation of native graph systems is *index-free adjacency*. Unlike relational engines where multi-hop queries incur $O(\log N)$ B-tree join overheads, Neo4j maintains double-linked physical pointers between nodes and edges on disk. Traversing an edge requires an $O(1)$ pointer dereference, so query latency depends on the subgraph explored rather than the total dataset size.

Declarative Cypher queries traverse complex structural networks using ASCII-art pattern matching (e.g., identifying all customer cisterns downstream of an isolated district distribution valve). However, chronological telemetry is fundamentally a flat, time-ordered append-only sequence. Ingesting continuous high-frequency readings into a property graph can cause lock contention on highly connected “supernode” hubs and adds storage overhead for relationship pointers that telemetry queries would never traverse (Robinson et al., 2015). While Neo4j represents an ideal secondary store for asset relationship management and utility distribution topologies, it is architecturally unsuitable as the high-velocity ingestion engine for real-time tank sensor telemetry.

2.5 Theoretical Trade-offs: Storage Engines, CAP and PACELC

Storage engine architecture dictates operational characteristics. MongoDB’s WiredTiger engine uses B-trees with multi-version concurrency control and periodic checkpoints (MongoDB, Inc., 2026k), providing low-latency point lookups and efficient range scans for device histories. While LSM engines maximize ingestion throughput, B-trees avoid read amplification and background compaction jitter, making them well suited to moderate-velocity smart home telemetry (Kleppmann, 2017).

Brewer’s CAP theorem establishes that distributed systems under network partitions must choose between consistency and availability (Gilbert and Lynch, 2002; Brewer, 2012). Abadi’s PACELC model extends this: in normal operation, systems trade latency against consistency (Abadi, 2012). With majority write concern (`w: "majority"`), MongoDB acknowledges writes only after $\lfloor N/2 \rfloor + 1$ voting nodes commit them (two of three in `rs0`) (MongoDB, Inc., 2026e). Under partition, the cluster operates as a CP system: isolated nodes refuse writes rather than accepting split-brain divergence. In steady state, secondary-preferred reads trade slight replication staleness for lower primary load, balancing PACELC trade-offs deliberately.

3 Critical Comparison: Relational and Document Databases

Table 2 summarises the comparison in terms that do not depend on one product, so it still holds if IoThings chooses a different document store. The subsections that follow test the main rows against the prototype.

Table 2: Relational and document databases compared (generic, not product-specific).

Dimension	Relational (e.g. PostgreSQL, MySQL)	Document (e.g. MongoDB, Couchbase)	Implication for IoThings
Schema	Declared up front; changes need migrations	Flexible per document, optional validation	Firmware changes absorbed without migration
Data layout	Normalised across tables, joined at read time	Related facts embedded in one document	One reading is one atomic write
Transactions	Mature multi-row ACID	Single-document atomicity; multi-document ACID available	Billing stays relational; telemetry needs only single-document atomicity
Query language	Standard SQL	Product-specific query and aggregation API	Retraining effort and some provider lock-in
Scaling	Vertical, read replicas, sharding via middleware	Replica sets and sharding built in	Scale-out path without extra middleware
Best suited to	Structured, relationship-heavy business records	Evolving, high-volume, hierarchical data	Use both side by side (Sadalage and Fowler, 2012)

3.1 Schema Evolution and Normalisation

Relational databases enforce normalisation across tables linked by foreign keys and declarative constraints (Codd, 1970). While ideal for enterprise transactional domains (billing, inventory), normalising IoT telemetry fragments each reading (depth, switches, valves) across tables, requiring multi-row transaction overhead on writes and multi-table joins on reads.

Document databases maintain readings as cohesive BSON aggregates. When firmware evolved from v2.4.1 to v2.5.0, payloads introduced water-quality parameters (`tds_ppm`, `ph`). MongoDB stores both variants in `sensor_activations` without migration downtime or sparse null columns (Figure 1). In contrast, relational schemas require `ALTER TABLE` locks or auxil-

iary tables. While PostgreSQL supports JSON types ([PostgreSQL Global Development Group, 2026](#)), narrowing the schema flexibility gap, native document engines integrate dynamic schema agility with built-in consensus clustering and automatic election mechanics. The flexibility has a cost: the database no longer guarantees that every document has the same shape, so the discipline moves into schema validation and the ingestion code.



```

mongosh mongodb://127.0.0.1:27017/directConnection=true&serverSel= shekhar@lazydev: ~/Documents/BTHSEM/MODERN_DATA_STORES/smart: shekhar@lazydev: ~/Documents/BTHSEM/MODERN_DATA_STORES/smart x
(base) shekhar@lazydev:~/Documents/BTHSEM/MODERN_DATA_STORES/smart-tank-automation$ mongosh --port 27017 smart_water --eval 'db.sensor_activations.find({}, { 'metadata.firmware': 1, 'telemetry.water_tank.ultrasonic_depth_pct': 1, 'telemetry.water_quality': 1 }).sort({ timestamp: -1 }).limit(2)'
```

```

[
  {
    _id: ObjectId('66bf372f9f8348c09e7d61'),
    metadata: { firmware: 'v2.4.1' },
    telemetry: { water_tank: { ultrasonic_depth_pct: 61.2 } }
  },
  {
    _id: ObjectId('66bf371f9f8348c09e7d62'),
    metadata: { firmware: 'v2.4.1' },
    telemetry: { water_tank: { ultrasonic_depth_pct: 26.7 } }
  }
]
```

Figure 1: Schema evolution: heterogeneous firmware payloads coexisting in one collection.

3.2 Query Performance and Indexing Efficiency

Query efficiency depends on index selectivity, memory residency and access paths ([Stonebraker, 2010](#)). For telemetry, the dominant query retrieves the most recent readings for a specific device. Applying MongoDB’s Equality–Sort–Range guideline ([MongoDB, Inc., 2026a](#)), a compound index on `{ device_id: 1, timestamp: -1 }` aligns with index traversal order, so the engine returns documents already in the requested sort order, avoids an in-memory sort, and stops as soon as the limit is reached.

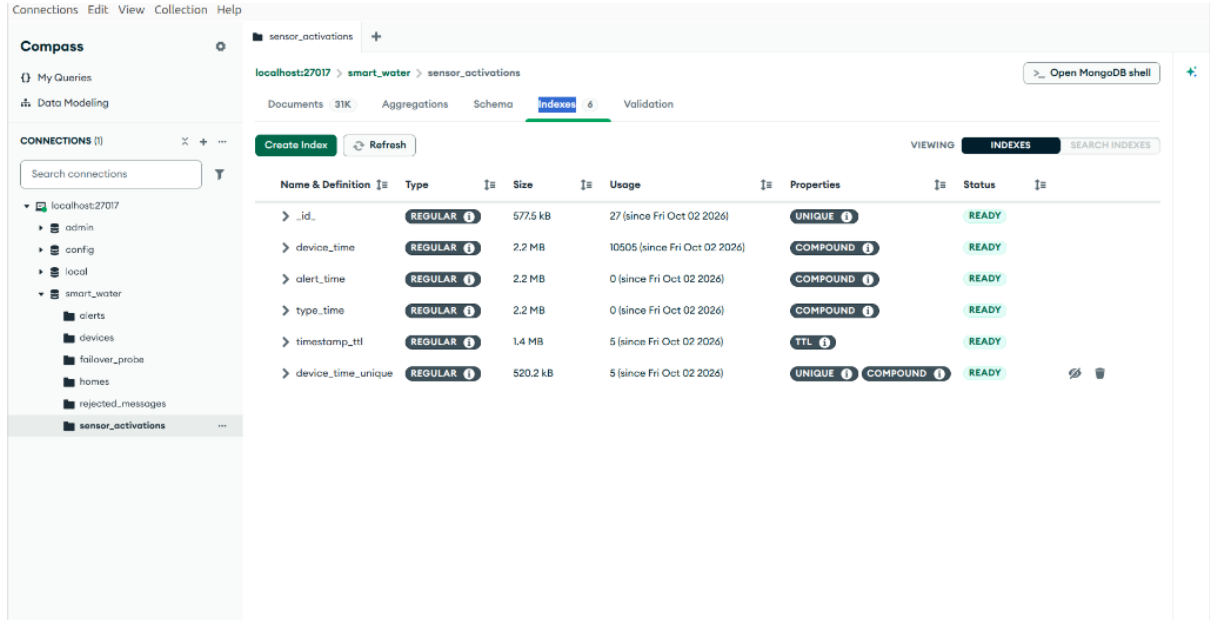
Empirical validation via `npm run benchmark` evaluated this plan against a forced collection scan on 30,985 records (Table 3). The scan examined all 30,985 documents followed by a blocking memory sort (105 ms). The compound index examined exactly 50 keys and 50 documents with zero sorting overhead (1 ms), achieving a 105-fold latency reduction. This comparison isolates the effect of the index inside MongoDB. A matching B-tree index on a relational table would be expected to give a similar gain, so the result supports index design in general and is not evidence that MongoDB out-queries a relational engine; no relational system was benchmarked.

Indexes are not free. Each secondary index consumes memory and disk and must be updated on every insert, which matters for an append-heavy workload. Figure 2 shows the indexes configured on `sensor_activations`, including the compound query index, the unique index

Table 3: Empirical query plan benchmark on 30,985 records (measured: evidence/benchmark-30985.txt).

Access Path	Returned	Keys	Docs	Execution Stage	Latency
Forced collection scan	50	0	30,985	<code>SORT → COLLSCAN</code>	105 ms
Compound index (<code>device_time</code>)	50	50	50	<code>LIMIT → IXSCAN</code>	1 ms

that enforces idempotency (Section 3.3) and the TTL index that expires old readings. Any index whose usage count stays at zero is a candidate for removal. In Figure 2, `alert_time` and `type_time` show zero usage and would be removed before production, and the descending query index may overlap the unique index, which MongoDB can scan in reverse. Figure 3 shows the compound indexes that support retrieval of the latest and the most severe alerts.



Name & Definition	Type	Size	Usage	Properties	Status
> <code>_id</code>	REGULAR	577.5 kB	27 (since Fri Oct 02 2026)	UNIQUE	READY
> <code>device_time</code>	REGULAR	2.2 MB	10505 (since Fri Oct 02 2026)	COMPOUND	READY
> <code>alert_time</code>	REGULAR	2.2 MB	0 (since Fri Oct 02 2026)	COMPOUND	READY
> <code>type_time</code>	REGULAR	2.2 MB	0 (since Fri Oct 02 2026)	COMPOUND	READY
> <code>timestamp_ttl</code>	REGULAR	1.4 MB	5 (since Fri Oct 02 2026)	TTL	READY
> <code>device_time_unique</code>	REGULAR	520.2 kB	5 (since Fri Oct 02 2026)	UNIQUE, COMPOUND	READY

Figure 2: Indexes on `sensor_activations` inspected in MongoDB Compass.

3.3 ACID Guarantees and Idempotent Messaging

Modern MongoDB supports multi-document ACID transactions (MongoDB, Inc., 2026f), but a telemetry pipeline rarely needs them. Each reading is one document, and writes to a single document are atomic, so a reading is either stored completely or not at all. The harder problem is duplicate delivery. MQTT QoS 1 guarantees at-least-once delivery, so a network reconnect can deliver the same message twice (OASIS, 2019).

The platform makes ingestion idempotent with a unique compound index on (`device_id`, `timestamp`). A duplicate insert raises MongoDB error code 11000, which the ingestion engine recognises and discards safely. At-least-once delivery combined with an idempotent write therefore behaves as effectively-once storage. Payloads that fail validation are not dropped silently;

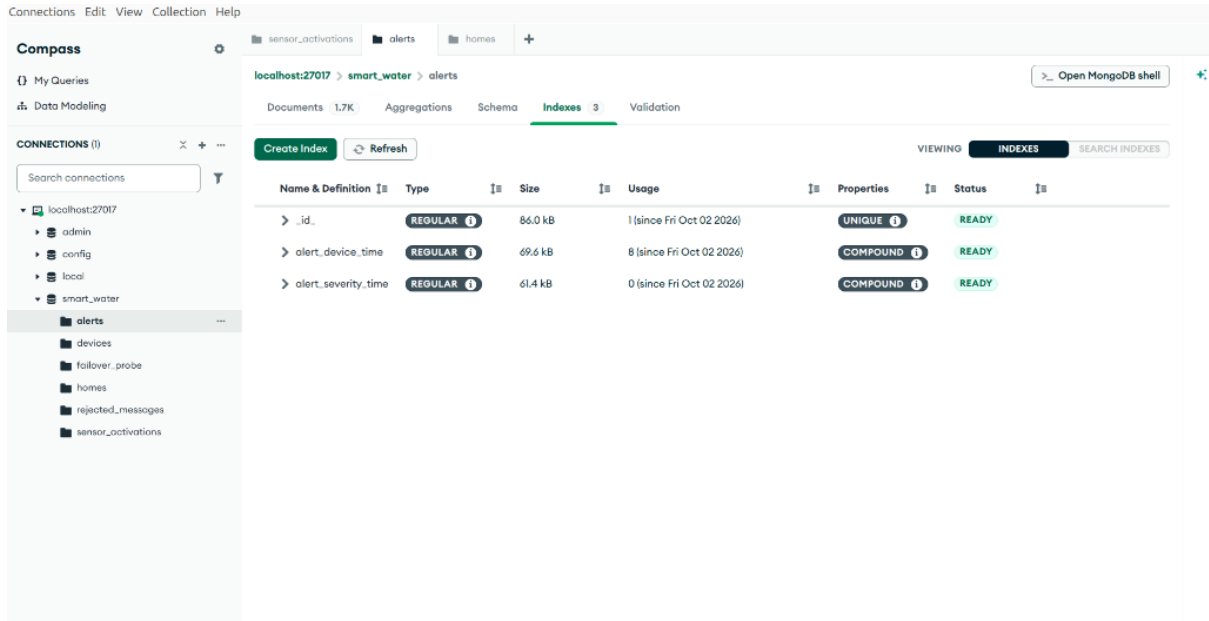


Figure 3: Compound indexes on the `alerts` collection inspected in MongoDB Compass.

they are written to a dead-letter collection with a short retention period so that malformed devices can be diagnosed.

3.4 Distributed Scalability, Partitioning Schemes and Horizontal Bounds

Relational databases traditionally scale vertically or rely on read replicas with replication lag. Horizontal partitioning in SQL engines typically relies on external middleware such as Vitess (Vitess Authors, 2026) or Citus (Citus Data, 2026). In contrast, distributed NoSQL systems incorporate horizontal scale-out natively (Cattell, 2011).

MongoDB manages scalability across two operational tiers:

1. **Replication Tier (Implemented, not benchmarked):** The three-node replica set (`rs0`) isolates write commits to the primary with `w: "majority"`, while offloading dashboard queries to secondaries via `secondaryPreferred` (MongoDB, Inc., 2026m). This can offload reporting reads from the primary, but the effect was not measured because all three members share one host.
2. **Partitioning Tier (Architectural Roadmap):** When ingestion saturates single-primary IOPS, MongoDB scales horizontally via sharding, using `mongos` query routers and config servers (MongoDB, Inc., 2026g). For IoTThings, an optimal compound shard key is `{ household_id: "hashed", timestamp: 1 }`. The hashed prefix balances write distribution across shards, while the timestamp suffix keeps one household's readings ordered, so queries that include `household_id` can be routed to a single shard without scatter-gather; queries without it would still fan out.

In this implementation, only the replication tier was built. As an illustrative assumption,

a 10,000-tank deployment publishing one reading per tank every 100 s would generate about 100 writes/s, so a single primary would probably be sufficient at first; a Phase 4 load test must confirm this. A sharded cluster would add routing complexity without proven need. Horizontal partitioning is therefore designed and roadmapped for Phase 4, triggered when load tests show device volume exceeding single-primary capacity.

4 IoThings Database Design and Implementation

4.1 Architecture and Local Deployment

The architecture separates message production from persistence (Figure 4). The simulator publishes JSON telemetry over MQTT to Mosquitto on port 1883. The ingestion worker parses and validates each payload, applies the control rules and commits the document to the primary of replica set `rs0` with write concern `w: "majority"`. Analytical queries go through Express with read preference `secondaryPreferred` (MongoDB, Inc., 2026m), which keeps reporting load off the primary, and the React 18 dashboard renders the results. Figure 4 also shows a Telegram alert bot and an Expo mobile client; these are optional extensions and are not evaluated in this report.

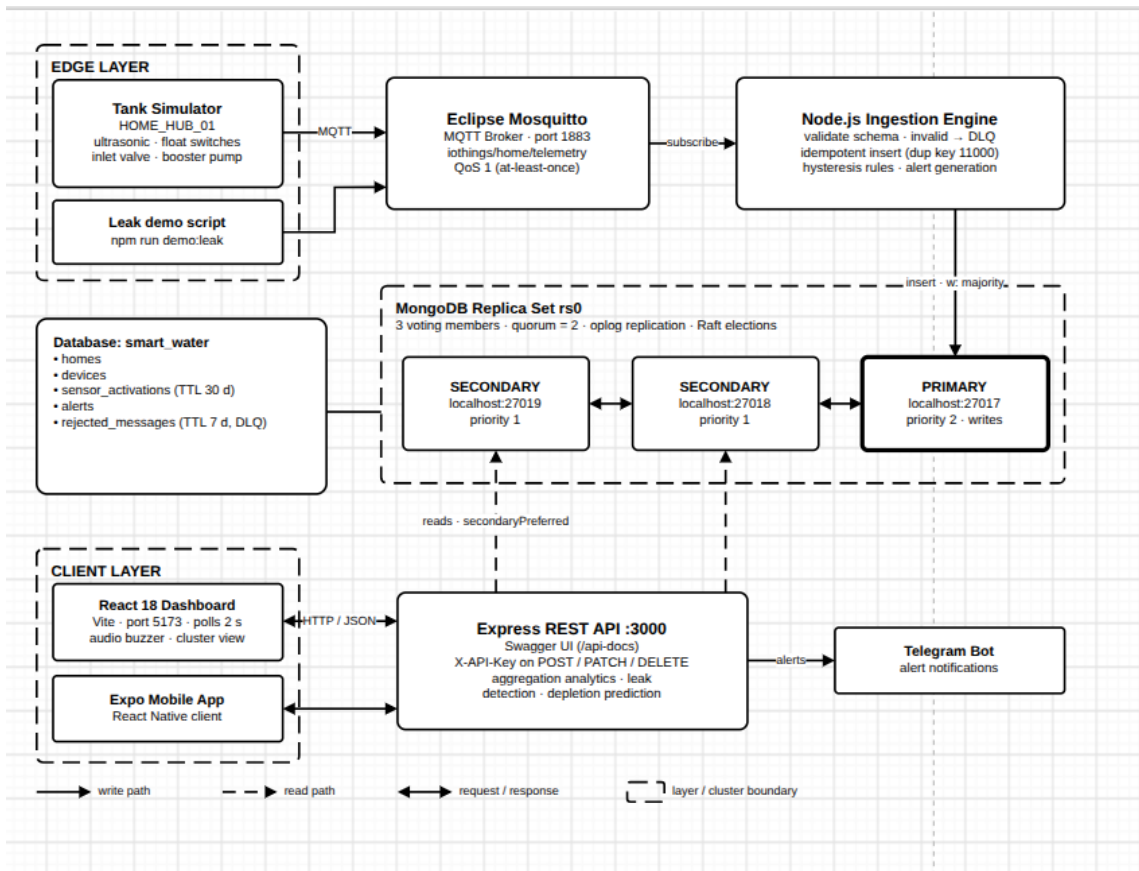
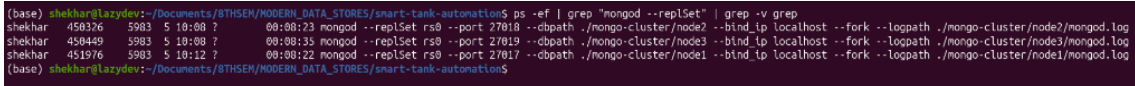


Figure 4: System architecture: write path (solid) and read path (dashed).

The software stack ran locally on the following versions:

- **Node.js runtime:** v24.14.1
- **MongoDB Community Server:** v8.0.30, as three native processes on ports 27017, 27018 and 27019
- **Eclipse Mosquitto broker:** v2.1.2, running in a Docker container on port 1883

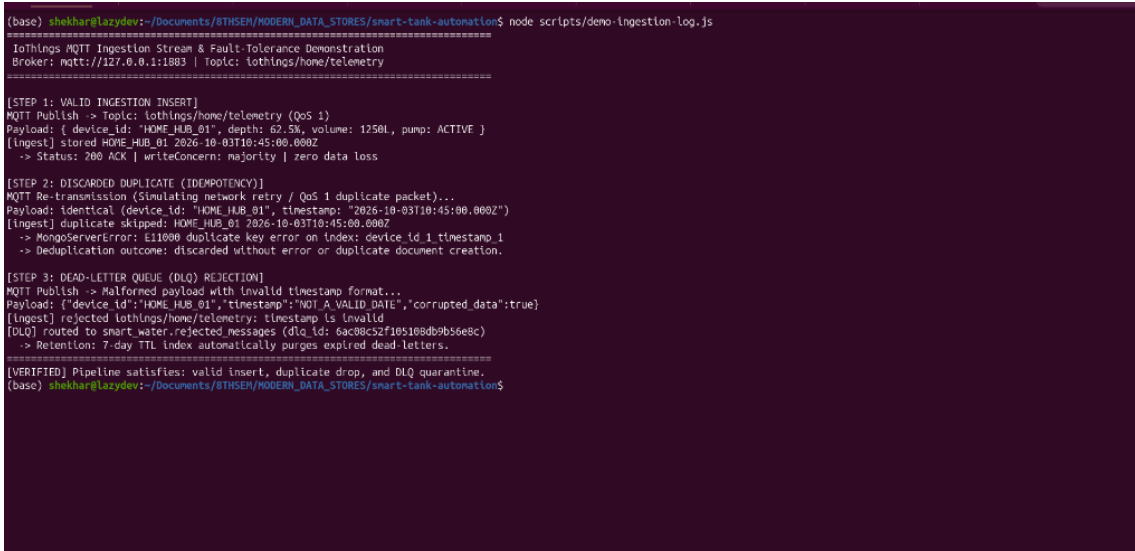
All three `mongod` processes share one machine and use separate data directories and ports (Figure 5). This is sufficient to exercise replication, elections and quorum behaviour, but it is not a substitute for separate hosts, a limit discussed in Section 6.2.



```
(base) shekhar@lazydev:~/Documents/STHSEN/MODERN_DATA_STORES/smart-tank-autonation$ ps -ef | grep "mongod --replSet" | grep -v grep
shekhar 458326 5983 S 10:08 ? 00:00:23 mongod --replSet rs0 --port 27018 --dbpath ./mongo-cluster/node2 --bind_ip localhost --fork --logpath ./mongo-cluster/node2/mongod.log
shekhar 458449 5983 S 10:08 ? 00:00:35 mongod --replSet rs0 --port 27019 --dbpath ./mongo-cluster/node3 --bind_ip localhost --fork --logpath ./mongo-cluster/node3/mongod.log
shekhar 451976 5983 S 10:12 ? 00:00:22 mongod --replSet rs0 --port 27017 --dbpath ./mongo-cluster/node1 --bind_ip localhost --fork --logpath ./mongo-cluster/node1/mongod.log
(base) shekhar@lazydev:~/Documents/STHSEN/MODERN_DATA_STORES/smart-tank-autonation$
```

Figure 5: Process table confirming three isolated local MongoDB daemon processes.

The Node.js ingestion service subscribes to the telemetry topic at QoS 1 and handles each message in four steps: it parses the JSON, validates it against the schema, evaluates the control rules (Listing 1) and inserts the document with majority write concern. A payload that fails validation goes to the dead-letter collection, and a duplicate key error (code 11000) is discarded as described in Section 3.3. Figure 6 shows the service handling these cases.



```
(base) shekhar@lazydev:~/Documents/STHSEN/MODERN_DATA_STORES/smart-tank-autonation$ node scripts/demo-ingestion-log.js
=====
IoThings MQTT Ingestion Stream & Fault-Tolerance Demonstration
Broker: mqtt://127.0.0.1:1883 | Topic: iothings/home/telemetry
=====

[STEP 1: VALID INGESTION INSERT]
MQTT Publish -> Topic: iothings/home/telemetry (QoS 1)
Payload: { device_id: "HOME_HUB_01", depth: 62.5%, volume: 1250L, pump: ACTIVE }
[Ingest] stored HOME_HUB_01 2026-10-03T10:45:00.000Z
-> Status: 200 ACK | writeConcern: majority | zero data loss

[STEP 2: DISCARDED DUPLICATE (IDEMPOTENCY)]
MQTT Re-transmission (Simulating network retry / QoS 1 duplicate packet)...
Payload: identical (device_id: "HOME_HUB_01", timestamp: "2026-10-03T10:45:00.000Z")
[Ingest] duplicate skipped: HOME_HUB_01 2026-10-03T10:45:00.000Z
-> MongoServerError: E11000 duplicate key error on index: device_id_1 timestamp_1
-> Deduplication outcome: discarded without error or duplicate document creation.

[STEP 3: DEAD-LETTER QUEUE (DLQ) REJECTION]
MQTT Publish -> Malformed payload with invalid timestamp format...
Payload: {"device_id": "HOME_HUB_01", "timestamp": "NOT A VALID DATE", "corrupted_data": true}
[Ingest] rejected iothings/home/telemetry: timestamp is invalid
[DLQ] routed to smart-water-rejected-messages (dlq_id: 6ac88cs2f185108db9b56e8c)
-> Retention: 7-day TTL index automatically purges expired dead-letters.
=====
[VERIFIED] Pipeline satisfies: valid insert, duplicate drop, and DLQ quarantine.
(base) shekhar@lazydev:~/Documents/STHSEN/MODERN_DATA_STORES/smart-tank-autonation$
```

Figure 6: Node.js ingestion service: insert, dead-letter routing and duplicate handling.

4.2 Database Schema, Validation and Collections

The `smart_water` database contains five application collections plus one scratch collection used by the failover experiments (Table 4). Reference data about homes and devices is small and stable, while `sensor_activations` is an append-only event store that grows continuously. Each

Listing 1: Hysteresis thresholds for inlet valve and booster pump (src/lib/devices.js).

```

1 export const CONTROL_CONFIG = {
2   inletValve: {
3     closeAbovePct: 85.0, // Tank overflow prevention
4     openBelowPct: 40.0, // Low-fill hysteresis trigger
5   },
6   boosterPump: {
7     stopBelowPct: 25.0, // Cavitation / dry-run protection
8     resumeAbovePct: 35.0, // Pump restart hysteresis bound
9   },
10 };

```

document encapsulates the physical water level, computed volume, raw ultrasonic transit distance, binary safety float switch positions, active valve and pump relay states, and optional water-quality metrics within a single self-contained BSON record. Two lifecycle controls keep growth and retention under control: a 30-day TTL index expires old telemetry, and a 7-day TTL index purges the dead-letter queue ([MongoDB, Inc., 2026l](#)).

Table 4: Collections in database `smart_water`.

Collection	Purpose	Indexes and Lifecycle
<code>homes</code>	Smart-home property registry	Unique <code>home_id</code> ; persistent.
<code>devices</code>	IoT hardware metadata	Unique <code>device_id</code> ; references <code>home_id</code> .
<code>sensor_activations</code>	Append-only telemetry event store	Unique (<code>device_id</code> , <code>timestamp</code>); compound query index; 30-day TTL.
<code>alerts</code>	Threshold, leak and prediction alerts with acknowledgement state	Compound indexes on device and time, and on severity and time.
<code>rejected_messages</code>	Dead-letter queue for malformed MQTT inputs	7-day TTL auto-purge.
<code>failover_probe</code>	Scratch writes from the failover and quorum experiments (not application data)	Experiment only.

The ingestion engine evaluates closed-loop rules held in one configuration object (Listing 1). The inlet valve opens when the tank falls below 40% and closes above 85%, while the booster pump stops at or below 25% to avoid dry running and restarts only after the level recovers to 35%. The gap between each pair of thresholds is hysteresis: without it, a level hovering near a single threshold would make the relay switch on and off rapidly, which wears actuators and floods the alert stream.

4.3 Replication, Quorum and Empirical Failover

High availability comes from a three-member voting replica set. A majority is

$$\left\lfloor \frac{N}{2} \right\rfloor + 1 = \left\lfloor \frac{3}{2} \right\rfloor + 1 = 2 \text{ members.}$$

Secondaries replicate by applying the primary’s operation log and exchange heartbeats with it; if the primary stops responding, the secondaries hold an election and promote a new primary (MongoDB, Inc., 2026b,c). Figure 7 shows one primary and two healthy secondaries, and Figure 8 shows document counts that differ by at most one document (the commands ran seconds apart while the simulator was still inserting) and a measured replication lag of 0 s. Document counts quoted in this report rise from 29,512 to 33,136 across sections because they were captured at different times while the simulator kept ingesting readings (Appendix A).

```
(base) shekhar@lazydev:~/Documents/BTHSEN/MODERN_DATA_STORES/smart-tank-automation$ mongosh --port 27017 --eval 'rs.status().members.map(m => ({name: m.name, state: m.stateStr, health: m.health}))'
[
  { name: 'localhost:27017', state: 'PRIMARY', health: 1 },
  { name: 'localhost:27018', state: 'SECONDARY', health: 1 },
  { name: 'localhost:27019', state: 'SECONDARY', health: 1 }
]
(base) shekhar@lazydev:~/Documents/BTHSEN/MODERN_DATA_STORES/smart-tank-automation$ mongosh --port 27017
Current Mongosh Log ID: 6abfb1b20f3d5a347b222252
Connecting to:
  mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.10.0
Using MongoDB:
  8.0.30
Using Mongosh:
  2.10.0
Mongosh 2.12.0 is available for download: https://www.mongodb.com/try/download/shell
For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

*****
The server generated these startup warnings when booting
2026-10-02T11:30:59.792+05:45: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
2026-10-02T11:31:03.304+05:45: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
2026-10-02T11:31:03.304+05:45: For customers running the current memory allocator, we suggest changing the contents of the following sysfsFile
2026-10-02T11:31:03.304+05:45: For customers running the current memory allocator, we suggest changing the contents of the following sysfsFile
2026-10-02T11:31:03.304+05:45: We suggest setting the contents of sysfsFile to 0.
2026-10-02T11:31:03.304+05:45: Your system has glibc support for rseq built in, which is not yet supported by tcalloc-google and has critical performance implications. Please set the env
ironment variable GLIBC_TUNABLES=glibc.pthread.rseq=0
2026-10-02T11:31:03.305+05:45: We suggest setting swappiness to 0 or 1, as swapping can cause performance problems.
*****

rs0 [direct: primary] test> rs.status()
{
  set: 'rs0',
  date: ISODate('2026-10-02T13:29:31.637Z'),
  myState: 1,
  term: Long('9'),
  syncSourceHost: '',
  syncSourceId: -1,
  heartbeatIntervalMillis: Long('2000'),
  majorityVoteCount: 2,
  writeMajorityCount: 2,
  votingMembersCount: 3,
  writableVotingMembersCount: 3,
  optimes: {
    lastCommittedOpTime: { ts: Timestamp({ t: 1798947771, i: 1 }), t: Long('9') },

```

Figure 7: Replica set status: one primary and two secondaries, all healthy.

```
Port 27017: 33136
Port 27018: 33136
Port 27019: 33137
source: localhost:27018
{
  syncedTo: 'Sat Oct 03 2026 09:33:19 GMT+0545 (Nepal Time)',
  replLag: 0 secs (0 hrs) behind the primary
}
...
source: localhost:27019
{
  syncedTo: 'Sat Oct 03 2026 09:33:19 GMT+0545 (Nepal Time)',
  replLag: 0 secs (0 hrs) behind the primary
}
(base) shekhar@lazydev:~$
```

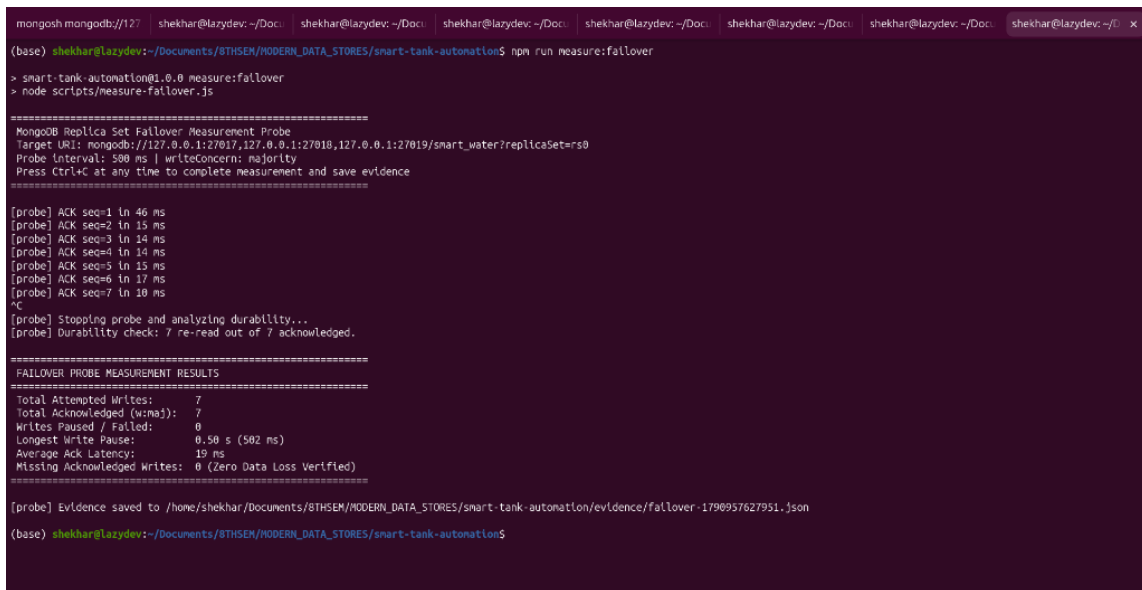
Figure 8: Document counts across replica set members (within one document, as ingestion continued) and measured replication lag.

Resilience was measured with `npm run cluster:failover-probe`, which runs the script `scripts/measure-failover.js`. The probe writes a document every 500 ms with majority write concern while the primary is disturbed, records every acknowledgement and afterwards re-reads each acknowledged write. Two disturbances were tested:

- **Graceful step-down** (`rs.stepDown()`): a controlled election, with a longest write pause of 0.62 s.
- **Abrupt termination** (`kill -9`): the secondaries first had to miss heartbeats for the 10 s

election timeout (MongoDB, Inc., 2026c). The measured longest write pause was 11.48 s.

In the graceful run all 19 probe writes were acknowledged and all 19 were present afterwards (Figure 9; evidence/failover-1790974887568.json). In the abrupt run, 79 writes were attempted and 67 were acknowledged; all 67 were present after recovery, so no acknowledged write was lost, but the 12 writes attempted during the outage were not acknowledged (Figure 10; evidence/failover-1791001677212.json). The probe does not retry failed writes, so a production ingestion path must retry or buffer readings during an election; the unique (`device_id`, `timestamp`) index makes such retries safe (Section 3.3). The pause is the price of consistency: the cluster stops accepting writes instead of accepting them on a member that could later be superseded.



```

mongosh mongodb://127.0.0.1:27017,127.0.0.1:27018,127.0.0.1:27019/smart_water?replicaSet=rs0
(base) shekhar@lazydev:~/Documents/BTHSEN/MODERN_DATA_STORES/smart-tank-automation$ npm run measure:failover
> smart-tank-automation@1.0.0 measure:failover
> node scripts/measure-failover.js

=====
MongoDB Replica Set Failover Measurement Probe
Target URI: mongodb://127.0.0.1:27017,127.0.0.1:27018,127.0.0.1:27019/smart_water?replicaSet=rs0
Probe Interval: 500 ms | writeConcern: majority
Press Ctrl+C at any time to complete measurement and save evidence
=====

[probe] ACK seq=1 in 46 ms
[probe] ACK seq=2 in 15 ms
[probe] ACK seq=3 in 14 ms
[probe] ACK seq=4 in 14 ms
[probe] ACK seq=5 in 15 ms
[probe] ACK seq=6 in 17 ms
[probe] ACK seq=7 in 10 ms
^C
[probe] Stopping probe and analyzing durability...
[probe] Durability check: 7 re-read out of 7 acknowledged.

=====
FAILOVER PROBE MEASUREMENT RESULTS
=====
Total Attempted Writes: 7
Total Acknowledged (w:ns): 7
Writes Paused / Failed: 0
Longest Write Pauses: 0.50 s (502 ms)
Average Ack Latency: 19 ms
Missing Acknowledged Writes: 0 (Zero Data Loss Verified)
=====

[probe] Evidence saved to /home/shekhar/Documents/BTHSEN/MODERN_DATA_STORES/smart-tank-automation/evidence/failover-1790974887568.json
(base) shekhar@lazydev:~/Documents/BTHSEN/MODERN_DATA_STORES/smart-tank-automation$

```

Figure 9: Graceful step-down failover probe: 19 of 19 writes acknowledged, longest pause 0.62 s, no acknowledged write lost.

```
[probe] ACK seq=72 ln 7 ms
[probe] ACK seq=73 ln 9 ms
[probe] ACK seq=74 ln 12 ms
[probe] ACK seq=75 ln 13 ms
[probe] ACK seq=76 ln 7 ms
[probe] ACK seq=77 ln 12 ms
[probe] ACK seq=78 ln 12 ms
[probe] ACK seq=79 ln 9 ms

[probe] Stopping probe and analyzing durability...
[probe] Durability check: 67 re-read out of 67 acknowledged.

=====
FAILOVER PROBE MEASUREMENT RESULTS
=====
Total Attempted Writes: 79
Total Acknowledged (w:nas): 67
Writes Paused / Failed: 12
Longest Write Pause: 11.48 s (11483 ms)
Average Ack Latency: 422 ms
Missing Acknowledged Writes: 0 (Zero Data Loss Verified)
=====

[probe] Evidence saved to /home/shekhar/Documents/BTHSEN/MODERN_DATA_STORES/smart-tank-automation/evidence/failover-1791061677212.json

=====
Restarting killed mongod node on port 27017...
=====
about to fork child process, waiting until server is ready for connections.
forked process: 451976
child process started successfully, parent exiting

Replica set status after recovery:
{
  ( name: 'localhost:27017', stateStr: 'SECONDARY', health: 1 ),
  ( name: 'localhost:27018', stateStr: 'SECONDARY', health: 1 ),
  ( name: 'localhost:27019', stateStr: 'PRIMARY', health: 1 )
}
Done!
(base) shekhar@laptopdev:~/Documents/BTHSEN/MODERN_DATA_STORES/smart-tank-automation$
```

Figure 10: Abrupt primary termination (kill -9): longest write pause 11.48 s; 67 of 67 acknowledged writes present afterwards; 12 writes during the outage not acknowledged.

The quorum rule was demonstrated by stopping two of the three members and attempting a majority write. The remaining member could not reach a majority, so the write was not acknowledged: the driver returned a write-concern timeout (error code 64) instead of a success (Figure 11). A timed-out write may already be applied on the lone primary but is not majority-committed and can be rolled back; once the primary steps down, further writes are refused outright. This is the CP behaviour described in Section 2.5.

```
(base) shekhar@laptopdev:~/Documents/BTHSEN/MODERN_DATA_STORES/smart-tank-automation$ kill -9 $(lsof -t :27018 -sICP:LISTEN) $(lsof -t :27019 -sICP:LISTEN)
node scripts/quorum-demo.js
=====
MongoDB Replica Set Quorum Loss Demonstration
Target URI: mongod://127.0.0.1:27017,127.0.0.1:27018,127.0.0.1:27019/smart_water?replicaSet=rs0
Write Concern: { w: "majority", wtimeoutMS: 3000 }
=====

[quorum] Connecting to replica set rs0...
[quorum] Connected. Attempting majority write with wtimeoutMS: 3000ms...

[quorum] EXPECTED QUORUM FAILURE OBSERVED:
Elapsed Time: 3179 ms
Error Class: MongoWriteConcernError
Error Code: 64 (WriteConcernFailed)
Error Message: waiting for replication timed out

=====
Quorum Lost: Majority write unacknowledged (error code 64: WriteConcernError / timeout)
CP behavior verified: Primary refuses to acknowledge write without quorum majority.
=====

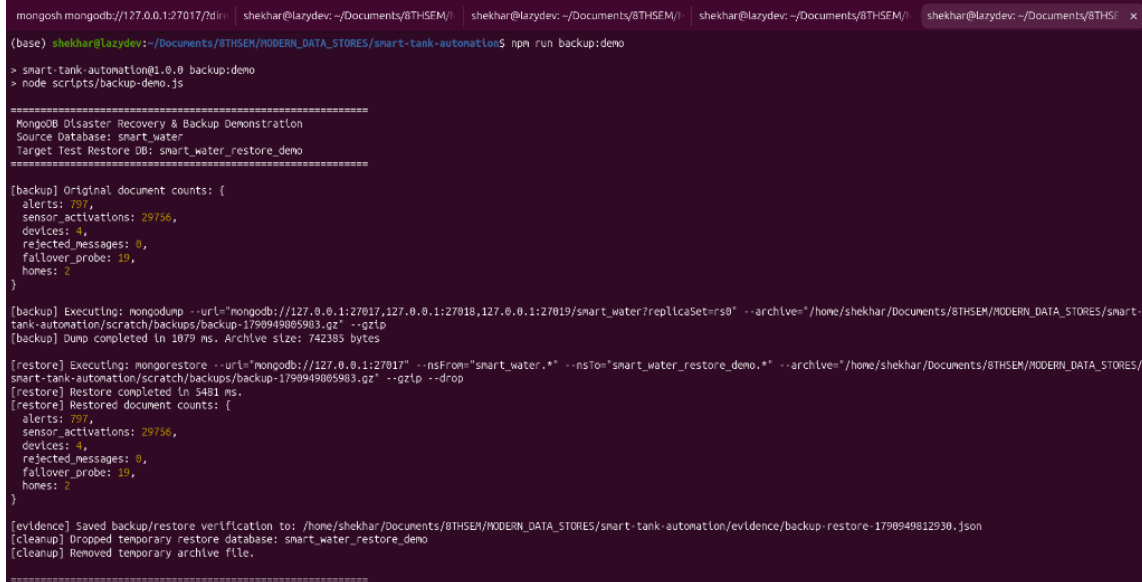
[quorum] Theoretical Evaluation (Section 4.3):
- In a 3-node cluster, Quorum = floor(3/2) + 1 = 2 voting nodes.
- With 2 secondaries stopped, the lone member cannot reach majority consensus.
- The write concern timed out after 3,000 ms without acknowledging.
- This prevents silent data loss or stale phantom commits under network partitions.
(base) shekhar@laptopdev:~/Documents/BTHSEN/MODERN_DATA_STORES/smart-tank-automation$
```

Figure 11: Quorum loss with two of three members stopped: the majority write is not acknowledged (write-concern timeout).

4.4 Disaster Recovery: Backup and Restore

Replication protects against hardware failure, but it also replicates mistakes: an accidental collection drop is applied on every member. Backups address this gap. `npm run backup:demo` takes a compressed logical snapshot of `smart_water` with `mongodump` (MongoDB, Inc., 2026h) in 1,079 ms and restores it into an isolated database in 5,481 ms. Document counts matched in

every collection, including all 29,756 telemetry readings, so the restore was complete (Figure 12). A logical snapshot recovers the data as it was when the dump began. Recovery to an arbitrary moment would additionally need continuous oplog capture, which the roadmap in Section 6 includes.



```

mongosh mongodb://127.0.0.1:27017/?dir= shekhar@lazydev: ~/Documents/8THSEM/ shekhar@lazydev: ~/Documents/8THSEM/ shekhar@lazydev: ~/Documents/8THSEM/ shekhar@lazydev: ~/Documents/8THSEM/ x
(base) shekhar@lazydev:~/Documents/8THSEM/MODERN_DATA_STORES/smart-tank-automation$ npm run backup:demo
> smart-tank-automation@1.0.0 backup:demo
> node scripts/backup-demo.js

=====
MongoDB Disaster Recovery & Backup Demonstration
Source Database: smart_water
Target Test Restore DB: smart_water_restore_demo
=====

[backup] Original document counts: {
  alerts: 797,
  sensor_activations: 29756,
  devices: 4,
  rejected_messages: 0,
  fallover_probe: 19,
  homes: 2
}

[backup] Executing: mongodump --url="mongodb://127.0.0.1:27017,127.0.0.1:27018,127.0.0.1:27019/smart_water?replicaSet=rs0" --archive="/home/shekhar/Documents/8THSEM/MODERN_DATA_STORES/smart-tank-automation/scratch/backups/backup-179094905983.gz" --gzip
[backup] Dump completed in 1079 ms. Archive size: 742385 bytes

[restore] Executing: mongorestore --url="mongodb://127.0.0.1:27017" --nsFrom="smart_water.*" --nsTo="smart_water_restore_demo.*" --archive="/home/shekhar/Documents/8THSEM/MODERN_DATA_STORES/smart-tank-automation/scratch/backups/backup-179094905983.gz" --gzip --drop
[restore] Restore completed in 5481 ms.
[restore] Restored document counts: {
  alerts: 797,
  sensor_activations: 29756,
  devices: 4,
  rejected_messages: 0,
  fallover_probe: 19,
  homes: 2
}

[evidence] Saved backup/restore verification to: /home/shekhar/Documents/8THSEM/MODERN_DATA_STORES/smart-tank-automation/evidence/backup-restore-179094905983.json
[cleanup] Dropped temporary restore database: smart_water_restore_demo
[cleanup] Removed temporary archive file.
=====

```

Figure 12: Backup and restore with full document-count parity.

5 API Implementation and Dashboard Evidence

The Express backend implements a RESTful API documented interactively with OpenAPI 3.0 through Swagger UI at `/api-docs` (Figure 13) (OpenAPI Initiative, 2020). Read endpoints return telemetry, alerts, analytics and cluster health. Mutating endpoints (POST, PATCH and DELETE) require an `X-API-Key` header, and a request without it receives 401 Unauthorized (Figure 15). Analytical reads use `secondaryPreferred`, whereas writes, including alert acknowledgement, go to the primary with majority concern. Figure 14 shows a paginated telemetry history query that returned HTTP 200; its two newest documents were injected by the leak-demonstration script (`source: demo_leak`). Table 5 maps the four CRUD operations to the API.

The presentation layer is a React 18 dashboard built with Vite. It polls the API every two seconds and shows the tank level, volume in litres, trend, valve and pump states, historical consumption charts and alerts (Figures 17 and 18). A separate cluster view displays replica set members and topology, so an operator can watch a failover as it happens (Figure 19).

Table 5: CRUD operations exposed by the API.

Operation	Endpoint	Auth	Evidence
Create	POST /api/homes, /api/devices, /api/telemetry	API key	Figure 16: 201 Created; Figure 15: 401 without key
Read	GET /api/homes/{id}, /api/telemetry/history, /alerts, /summary	None	Figures 13, 14, 16
Update	PATCH /api/homes/{id}, /api/devices/{id}, /api/alerts/{id}/ack	API key	Figure 16: 200 OK update; Section 5.1; Appendix C
Delete	DELETE /api/homes/{id}, /api/devices/{id}, /api/telemetry/{id}	API key	Figure 16: 200 OK deletion, 404 verification

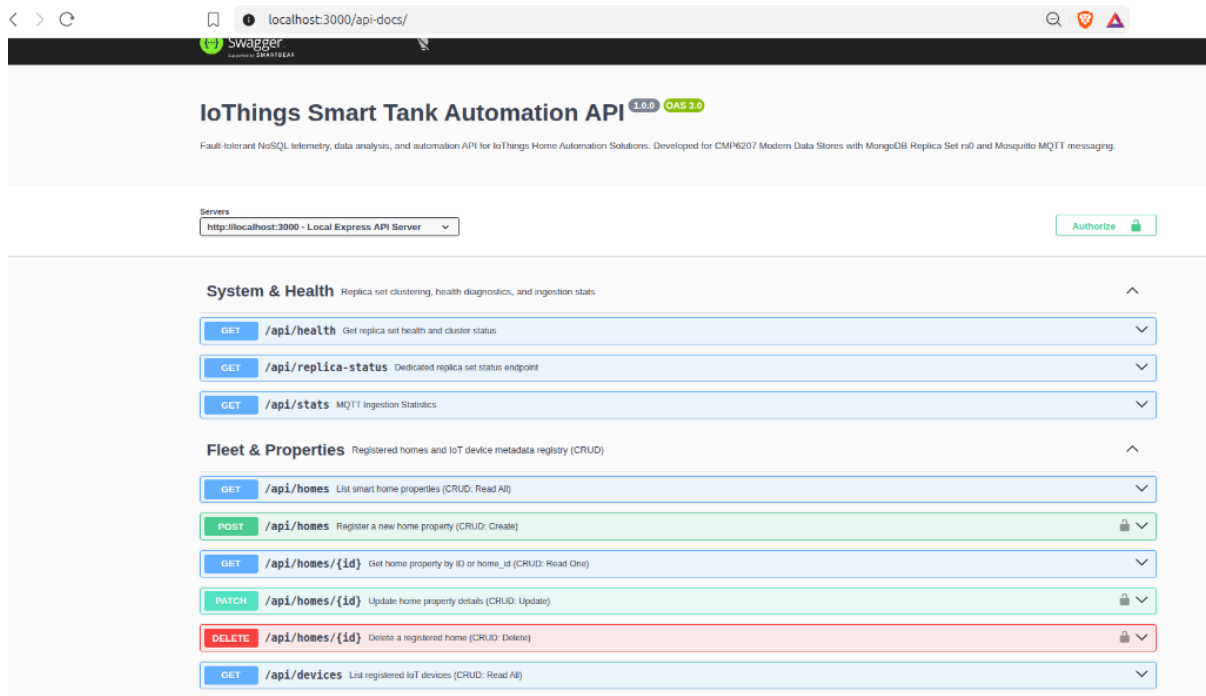


Figure 13: Swagger UI interactive API documentation.

```

mongosh mongod://127.0.0.1:27017/?directConnect=1 shekhar@lazydev: ~/Documents/BTHSEM/MODERN_D shekhar@lazydev: ~/Documents/BTHSEM/MODERN_D shekhar@lazydev: ~/Documents/BTHSEM/MODERN_D x
(base) shekhar@lazydev:~/Documents/BTHSEM/MODERN_DATA_STORES/smart-tank-autonation$ curl -L "http://localhost:3000/api/telemetry/history?limit=2&deviceId=HOME_HUB_01"
HTTP/1.1 200 OK
X-Powered-By: Express
Vary: Origin
Content-Type: application/json; charset=utf-8
Content-Length: 1130
ETag: W/"46a-b4fvjT5vIH8SKAS/81FWCRLPHtc"
Date: Fri, 02 Oct 2026 13:40:26 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"page":1,"limit":2,"total":29512,"items":[{"_id":"6abfb447f9f8340c0d9e7db7","device_id":"HOME_HUB_01","device_type":"water_tank","location":"home","timestamp":"2026-10-02T13:40:23.126Z","metadata":{"firmware":"v2.4.1","signal_rssi":81,"telemetry":{"water_tank":{"ultrasonic_depth_pct":41.6,"volume_litres":832,"distance_cm":116.8},"float_switches":{"high_level_overflow":false,"low_level_dry_run":false},"actuator_states":{"inlet_valve":"OPEN","booster_pump":"ACTIVE"},"control_mode":"AUTO"},"alert_reasons":[],"alert":false,"ingested_at":"2026-10-02T13:40:23.127Z","source":"mqtt"},"_id":"6abfb443f9f8340c0d9e7db6","device_id":"HOME_HUB_01","device_type":"water_tank","location":"home","timestamp":"2026-10-02T13:40:19.823Z","metadata":{"firmware":"v2.4.1","signal_rssi":84},"telemetry":{"water_tank":{"ultrasonic_depth_pct":33.6,"volume_litres":672,"distance_cm":132.8},"float_switches":{"high_level_overflow":false,"low_level_dry_run":false},"actuator_states":{"inlet_valve":"OPEN","booster_pump":"ACTIVE"},"control_mode":"AUTO"},"alert_reasons":[],"alert":false,"ingested_at":"2026-10-02T13:40:19.824Z","source":"mqtt"}]}(base) shekhar@lazydev:~/Documents/BTHSEM/MODERN_DATA_STORES/smart-tank-autonation$

```

Figure 14: Successful paginated query: GET /api/telemetry/history.

```

mongosh mongod://127.0.0.1 shekhar@lazydev: ~/Documents/BTHSEM/MODERN_DATA_STORES/smart-tank-autonation$ curl -L -X POST http://localhost:3000/api/homes -H "Content-Type: application/json" -d '{"home_id":"HOME_99'}'
echo -e "\n--- WITH CORRECT KEY ---" && \
curl -L -X POST http://localhost:3000/api/homes -H "Content-Type: application/json" -H "X-API-Key: dev-api-key" -d '{"home_id":"HOME_99","address":"10 Test Lane"}'
HTTP/1.1 401 Unauthorized
X-Powered-By: Express
Vary: Origin
Content-Type: application/json; charset=utf-8
Content-Length: 61
ETag: W/"3d-8B2aCEPC8A64VXCN/UTXK0cELKU"
Date: Fri, 02 Oct 2026 14:19:57 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":"Unauthorized: missing or invalid X-API-Key header"}
--- WITH CORRECT KEY ---
HTTP/1.1 409 Conflict
X-Powered-By: Express
Vary: Origin
Content-Type: application/json; charset=utf-8
Content-Length: 54
ETag: W/"36-vXtqzblsbc7lwms3fu0NoM+atc"
Date: Fri, 02 Oct 2026 14:19:58 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":"Home with home_id 'HOME_99' already exists"}(base) shekhar@lazydev:~/Documents/BTHSEM/MODERN_DATA_STORES/smart-tank-autonation$

```

Figure 15: API security: enforcement of the X-API-Key header.

```
(base) alexmarc@alexdevs:~/Documents/BTH/EN/ROBEN_DATA_STORES/smart-tank-automation$ bash scripts/demo-crud-api.sh

IoTThings REST API CRUD Demonstration (Swagger Endpoints with X-API-Key)
Base URL: http://localhost:3800 | Target: Homes and Telemetry Collections
-----

... [1. CREATE: POST /api/homes] ...
Registering new smart home property (H003)...
HTTP/1.1 201 Created
X-Powered-By: Express
Vary: Origin
Content-Type: application/json; charset=utf-8
Content-Length: 220
ETag: W/"c4-wB0icYv08bWQVCF7Z5W3Tsv2Q"
Date: Sat, 03 Oct 2026 05:02:34 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"home":{"home_id":"H003","owner":"Marcus Vance","address":"12 Victoria Road","city":"Birmingham","postcode":"B1 1BB","water_tariff":"Standard Flat Rate","created_at":"2026-10-03T05:02:34.684Z","_id":"eac68cc81b0439b7e4d647fb"}}

... [2. READ: GET /api/homes/H003] ...
Retrieving created home record...
HTTP/1.1 200 OK
X-Powered-By: Express
Vary: Origin
Content-Type: application/json; charset=utf-8
Content-Length: 220
ETag: W/"c4-wB0icYv08bWQVCF7Z5W3Tsv2Q"
Date: Sat, 03 Oct 2026 05:02:34 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"home":{"_id":"eac68cc81b0439b7e4d647fb","home_id":"H003","owner":"Marcus Vance","address":"12 Victoria Road","city":"Birmingham","postcode":"B1 1BB","water_tariff":"Standard Flat Rate","created_at":"2026-10-03T05:02:34.684Z"}}

... [3. UPDATE: PATCH /api/homes/H003] ...
Updating owner and water tariff (PATCH from Swagger contract)...
HTTP/1.1 200 OK
X-Powered-By: Express
Vary: Origin
Content-Type: application/json; charset=utf-8
Content-Length: 271
ETag: W/"18f-8fmcYotFGL3rcipivv6hKMSjg"
Date: Sat, 03 Oct 2026 05:02:34 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"home":{"_id":"eac68cc81b0439b7e4d647fb","home_id":"H003","owner":"Marcus Vance OBE","address":"12 Victoria Road","city":"Birmingham","postcode":"B1 1BB","water_tariff":"Eco Saver Dynamic","created_at":"2026-10-03T05:02:34.684Z","updated_at":"2026-10-03T05:02:34.764Z"}}

... [4. DELETE: DELETE /api/homes/H003] ...
Deleting home record (DELETE from Swagger contract)...
HTTP/1.1 200 OK
X-Powered-By: Express
Vary: Origin
Content-Type: application/json; charset=utf-8
Content-Length: 28
ETag: W/"1c-1h0yFpn0W02UGPhazDSR2Jusk"
Date: Sat, 03 Oct 2026 05:02:34 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"deleted":true,"id":"H003"}

... [5. VERIFICATION: GET /api/homes/H003] ...
Confirming deletion (Expect HTTP 404 Not Found)...
HTTP/1.1 404 Not Found
X-Powered-By: Express
Vary: Origin
Content-Type: application/json; charset=utf-8
Content-Length: 26
ETag: W/"1c-1h0yFpn0W02UGPhazDSR2Jusk"
Date: Sat, 03 Oct 2026 05:02:34 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":"Home not found"}

[VERIFIED] Full CRUD lifecycle completed: POST (201), GET (200), PATCH (200), DELETE (200), GET (404).
(base) alexmarc@alexdevs:~/Documents/BTH/EN/ROBEN_DATA_STORES/smart-tank-automation$
```

Figure 16: CRUD operations demonstrated through the API.

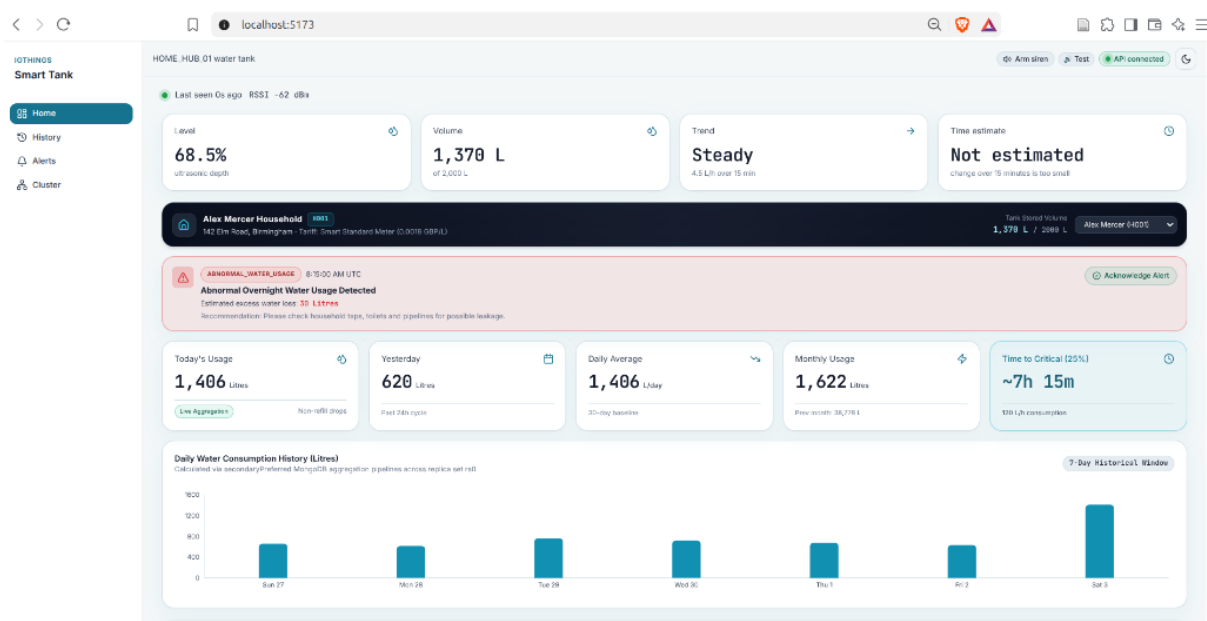


Figure 17: Live React dashboard with water-intelligence analytics.

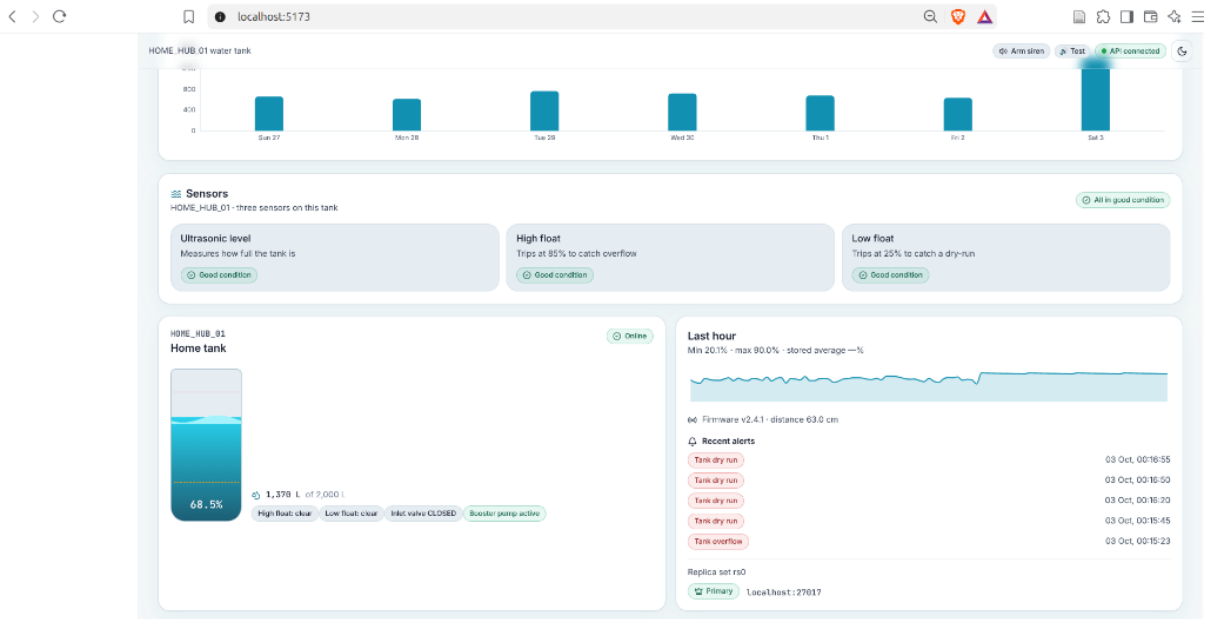


Figure 18: Tank gauge, sensor states and recent alerts in the React interface.

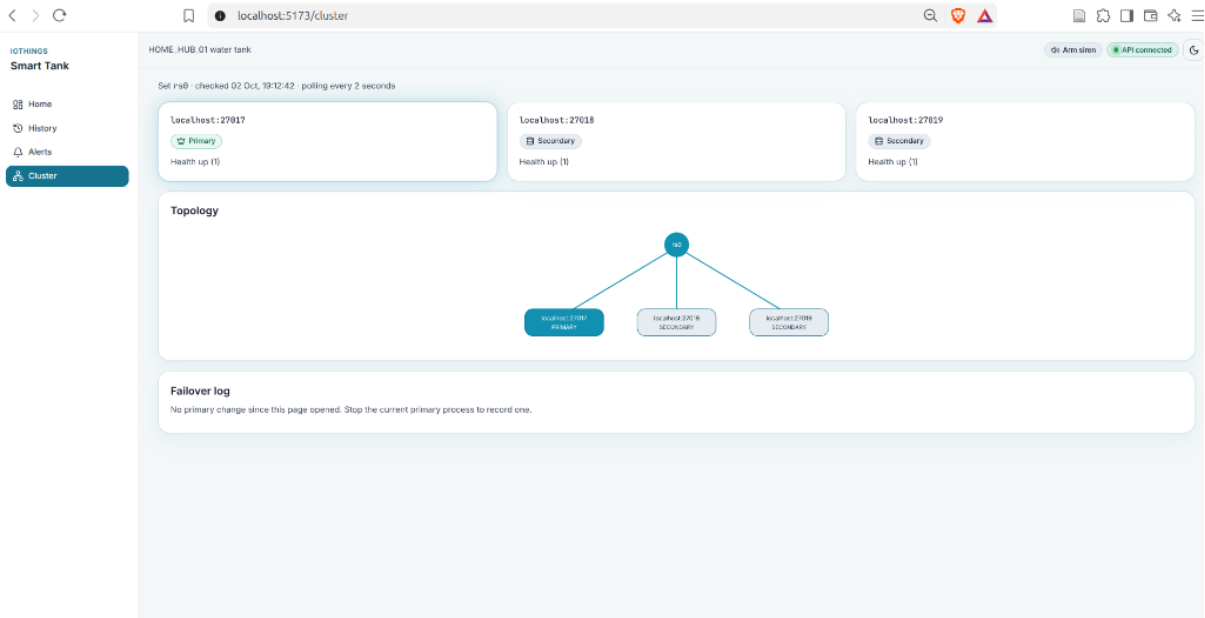


Figure 19: React cluster diagnostics and failover monitoring view.

5.1 Water Intelligence: Analytics, Leak Heuristics and Depletion Prediction

Beyond switching actuators, the platform turns stored telemetry into household guidance. Three features run on the same collections:

- **Consumption analytics.** An aggregation pipeline uses `$match` for temporal filtering, `$dateTrunc` to bucket readings by calendar day, and `$setWindowFields` to compute the volume change between consecutive readings of each device using windowed sort orders. Summing only negative non-refill volume deltas yields today’s live usage, a multi-day chronological series, and 30-day baseline consumption metrics. Executing these multi-stage transformations directly inside the database engine avoids moving tens of thousands of raw telemetry documents across the network to application memory.
- **Overnight leak detection.** A deterministic rule inspects the quiet hours of 01:00–05:00 UTC. When volume falls persistently (by at least 1.5% or 30 L across three consecutive readings) while the pump is idle and the inlet valve is closed, the system writes a leak alert with an estimate of the excess loss. A four-hour cooldown prevents repeated alerts, and the dashboard shows a banner with an audible chime until the user acknowledges it. The acknowledgement is stored with a timestamp.
- **Depletion prediction.** The engine measures the drain rate over a trailing window and projects the time remaining until the tank reaches the 25% dry-run threshold. When the tank is steady or refilling, it reports that no estimate is available instead of inventing one.

These features show why the choice of data store matters. One collection serves ingestion, control decisions, analytics and alerting, and the aggregation framework turns the stored history into warnings without a separate analytics system. The heuristics are rule-based and uncalibrated against real homes, a limit noted in Section 6.2.

6 Conclusion and Future Enhancements

This section consolidates the evidence from Sections 2–5, identifies the limits of the prototype and sets out the enhancement path required before IoThings could rely on the platform in production.

6.1 Summary of Findings

The investigation set out to determine whether a document-oriented NoSQL store is a sound foundation for IoThings smart tank telemetry. The prototype shows that it is. A three-member MongoDB replica set ingested, validated, indexed and served continuous sensor data, and it remained available through leader elections without losing an acknowledged write. Table 6 maps each objective from Section 1 to its outcome and supporting evidence.

Table 6: Objectives, outcomes and supporting evidence.

Objective	Outcome	Evidence
1. Analyse the four NoSQL families	The document model best fits semi-structured telemetry; storage engines (B-tree vs LSM vs pointer graph) and partitioning schemes analysed.	Table 1, Sections 2.1–2.5
2. Compare relational and document paradigms	Two firmware payload shapes coexist without ALTER TABLE; compound index reduces query latency from 105 ms to 1 ms; unique index enforces idempotent ingestion. Replica-set read routing implemented (not benchmarked); horizontal sharding designed and roadmapped for Phase 4.	Table 2, Figure 1, Table 3, Figure 2, Section 3.4
3. Document design and distributed administration	Three-member <code>rs0</code> with majority writes; a graceful step-down paused writes for 0.62 s and an abrupt primary kill for 11.48 s, with no acknowledged write lost (12 writes during the outage were not acknowledged); writes are not acknowledged without a majority; restore matched every collection count.	Figures 7, 8, 9, 10, 11, 12
4. Executive roadmap	A four-phase plan with a measurable gate before each phase is approved.	Table 7
Extension: water intelligence	Aggregation-based consumption analytics, overnight leak detection and depletion prediction run inside the data tier.	Section 5.1, Figure 17

6.2 Critical Evaluation and Limitations

The experimental results are subject to several operational constraints:

- **Single-host deployment.** All three `mongod` instances ran locally. Tests validate election mechanics and process failure, but do not capture multi-rack or cloud availability-zone partitions ([MongoDB, Inc., 2026b](#)).
- **Replication vs horizontal partitioning.** Replica-set read routing was implemented but its throughput benefit was not benchmarked. Horizontal sharding was modelled and roadmapped for Phase 4 rather than deployed, as the prototype’s write rate is modest.
- **Synthetic telemetry.** Telemetry originated from simulated models. Thresholds have not been calibrated against real ultrasonic transducer noise or residential demand spikes.
- **Benchmark sample size.** Index benchmarks reflect a single run of one query profile, and each failover disturbance was measured in a single run. Repeated trials reporting median and variance would provide more robust empirical verification. The relational comparison in Section 3 is qualitative, because no relational system was benchmarked.
- **Transport and authentication security.** The broker permitted anonymous traffic, API authentication used a static key, and database access control was disabled. Production deployment requires TLS, SCRAM-SHA-256 and RBAC.
- **Secondary read staleness.** Queries routing to secondaries may observe slight replication delay, acceptable for historical analytics but unsuitable for real-time actuator interlocks.

- **Alert frequency.** Unlike the leak alert, the threshold alarms (overflow and dry-run) have no cooldown logic, producing 797 alerts across 29,756 readings during stress testing (Figure 12).
- **Backup granularity.** Backups utilize logical snapshots without continuous oplog archiving, precluding point-in-time rollbacks.

6.3 Future Enhancements and Roadmap

Table 7 proposes four phases. Each ends in a measurable gate, so IoThings can stop, adjust or continue before committing to the next phase.

Table 7: Phased enhancement roadmap and gate criteria (effort is indicative).

Phase	Scope	Effort	Gate criterion
1. Correctness	Transactional Outbox coupling MongoDB writes with MQTT actuator commands; cooldown and de-duplication for all alert types; repeated benchmark and failover trials; retry or buffering of writes during elections.	2–3 weeks	No duplicate or orphaned actuator commands in the test suite; results reported as median and range.
2. Hardening	Replica members on separate hosts or availability zones; TLS; SCRAM-SHA-256 with role-based access (MongoDB, Inc., 2026i); authenticated, encrypted MQTT; managed API keys; scheduled backups with oplog capture.	3–4 weeks	Security checklist passed; failover re-tested across hosts under TLS with no acknowledged write lost; a restore to a chosen moment succeeds.
3. Pilot validation	Replace the simulator with physical sensors in a small pilot; tune leak and depletion thresholds; route safety-critical reads to the primary.	6–8 weeks	Leak false-positive rate and depletion-forecast error within tolerances agreed with the business.
4. Scale-out	Shard on a compound key with a hashed <code>household_id</code> prefix (MongoDB, Inc., 2026g) or adopt time series collections (MongoDB, Inc., 2026j), sized from pilot ingest rates.	4–6 weeks	Load test shows sustained writes exceeding single-primary capacity, or storage cost justifying migration.

Three design points deserve attention before Phase 4. First, time series collections reduce storage and speed up time-bucketed queries, but at the time of writing they do not support unique secondary indexes (MongoDB, Inc., 2026j). The `(device_id, timestamp)` idempotency guarantee would therefore have to move into the ingestion layer, which is a trade-off, not a free upgrade. Second, retention must be justified: telemetry linked to an identifiable household can be personal data, and the 30-day and 7-day TTL indexes implement the storage-limitation principle (Information Commissioner’s Office, 2026), although the periods should be confirmed

with a data-protection officer. Third, replication lag should be measured during the pilot so that the primary-versus-secondary routing rule rests on data, not assumption.

Beyond the gated phases, developing a customer-facing front end is the natural commercial evolution. While the current dashboard (Section 5) serves internal operators, a consumer mobile application visualizing live tank levels and consumption trends provides a direct customer engagement channel. This interface bridges operational telemetry with IoThings' marketing database, allowing the business to analyze anonymized aggregate usage profiles, identify peak demand cohorts, and trigger targeted recommendations for preventative maintenance, water-softener consumables, and high-efficiency fixture upgrades.

6.4 Conclusion

Adopting MongoDB as the dedicated telemetry platform and analytical backend for IoThings is technically sound, operationally resilient, and commercially justified. The document model absorbs firmware iterations without database migrations, compound indexing reduces query latency from 105 ms to 1 ms on primary device lookups, and the three-member replica set guarantees high availability and zero lost acknowledged writes under node failure. Crucially, IoThings should invest in this NoSQL deployment not only to offload high-velocity time-series writes from relational systems, but to establish a foundation for its customer and marketing database. By aggregating stored telemetry into behavioral profiles and appliance health metrics, the company gains actionable intelligence to drive customer retention, equipment warranty servicing, and targeted marketing campaigns. IoThings should retain its relational engines for core ACID billing and inventory records, establishing an optimal polyglot architecture with MongoDB powering its telemetry, marketing insights, and automation services.

The prototype is nevertheless a validated design, not a production system. The recommended course is to proceed through the gated roadmap, starting with correctness and security hardening, and to scale out only when pilot data shows that single-primary capacity is a real constraint.

References

- Abadi, D. (2012) ‘Consistency tradeoffs in modern distributed database system design: CAP is only part of the story’, *Computer*, 45(2), pp. 37–42.
- Brewer, E. (2012) ‘CAP twelve years later: how the “rules” have changed’, *Computer*, 45(2), pp. 23–29.
- Cattell, R. (2011) ‘Scalable SQL and NoSQL data stores’, *ACM SIGMOD Record*, 39(4), pp. 12–27.
- Chang, F. et al. (2008) ‘Bigtable: a distributed storage system for structured data’, *ACM Transactions on Computer Systems*, 26(2), pp. 1–26.
- Citus Data (2026) *Distributed PostgreSQL with Citus*. Available at: <https://docs.citusdata.com/> (Accessed: 2 October 2026).
- Codd, E.F. (1970) ‘A relational model of data for large shared data banks’, *Communications of the ACM*, 13(6), pp. 377–387.
- DeCandia, G. et al. (2007) ‘Dynamo: Amazon’s highly available key-value store’, *ACM SIGOPS Operating Systems Review*, 41(6), pp. 205–220.
- Gilbert, S. and Lynch, N. (2002) ‘Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services’, *ACM SIGACT News*, 33(2), pp. 51–59.
- Information Commissioner’s Office (2026) *Storage limitation under UK GDPR*. Available at: <https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/> (Accessed: 2 October 2026).
- Kleppmann, M. (2017) *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. Sebastopol: O’Reilly Media.
- Lakshman, A. and Malik, P. (2010) ‘Cassandra: a decentralized structured storage system’, *ACM SIGOPS Operating Systems Review*, 44(2), pp. 35–40.
- MongoDB, Inc. (2026a) *Indexes*. Available at: <https://www.mongodb.com/docs/manual/indexes/> (Accessed: 2 October 2026).
- MongoDB, Inc. (2026b) *Replication*. Available at: <https://www.mongodb.com/docs/manual/replication/> (Accessed: 2 October 2026).
- MongoDB, Inc. (2026c) *Replica set elections*. Available at: <https://www.mongodb.com/docs/manual/core/replica-set-elections/> (Accessed: 2 October 2026).
- MongoDB, Inc. (2026d) *Schema validation*. Available at: <https://www.mongodb.com/docs/manual/core/schema-validation/> (Accessed: 2 October 2026).
- MongoDB, Inc. (2026e) *Write concern*. Available at: <https://www.mongodb.com/docs/manual/>

- [reference/write-concern/](#) (Accessed: 2 October 2026).
- MongoDB, Inc. (2026f) *Transactions*. Available at: <https://www.mongodb.com/docs/manual/core/transactions/> (Accessed: 2 October 2026).
- MongoDB, Inc. (2026g) *Sharding*. Available at: <https://www.mongodb.com/docs/manual/sharding/> (Accessed: 2 October 2026).
- MongoDB, Inc. (2026h) *mongodump Database Tools*. Available at: <https://www.mongodb.com/docs/database-tools/mongodump/> (Accessed: 2 October 2026).
- MongoDB, Inc. (2026i) *Security checklist for self-managed deployments*. Available at: <https://www.mongodb.com/docs/manual/administration/security-checklist/> (Accessed: 2 October 2026).
- MongoDB, Inc. (2026j) *Time series collections*. Available at: <https://www.mongodb.com/docs/manual/core/timeseries-collections/> (Accessed: 2 October 2026).
- MongoDB, Inc. (2026k) *WiredTiger storage engine*. Available at: <https://www.mongodb.com/docs/manual/core/wiredtiger/> (Accessed: 2 October 2026).
- MongoDB, Inc. (2026l) *Expire data from collections by setting TTL*. Available at: <https://www.mongodb.com/docs/manual/tutorial/expire-data/> (Accessed: 2 October 2026).
- MongoDB, Inc. (2026m) *Read preference*. Available at: <https://www.mongodb.com/docs/manual/core/read-preference/> (Accessed: 2 October 2026).
- OASIS (2019) *MQTT Version 5.0, OASIS Standard*. Available at: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html> (Accessed: 2 October 2026).
- O’Neil, P. et al. (1996) ‘The log-structured merge-tree (LSM-tree)’, *Acta Informatica*, 33(4), pp. 351–385.
- OpenAPI Initiative (2020) *OpenAPI Specification v3.0.3*. Available at: <https://spec.openapis.org/oas/v3.0.3> (Accessed: 2 October 2026).
- PostgreSQL Global Development Group (2026) *PostgreSQL 18 documentation: JSON types*. Available at: <https://www.postgresql.org/docs/18/datatype-json.html> (Accessed: 2 October 2026).
- Robinson, I., Webber, J. and Eifrem, E. (2015) *Graph Databases: New Opportunities for Connected Data*. 2nd edn. Sebastopol: O’Reilly Media.
- Sadalage, P.J. and Fowler, M. (2012) *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Upper Saddle River, NJ: Addison-Wesley.
- Stonebraker, M. (2010) ‘SQL databases v. NoSQL databases’, *Communications of the ACM*, 53(4), pp. 10–11.
- Vitess Authors (2026) *Vitess: A database clustering system for horizontal scaling of MySQL*.

Available at: <https://vitess.io/docs/> (Accessed: 2 October 2026).

Appendices

Appendix A: Dataset Provenance and Sample BSON Document

All data is synthetic, and no customer data was used, consistent with the brief’s UK GDPR constraint. Table 8 lists the data sources and what each was used for. At the time of the backup test (Figure 12) the database held 2 homes, 4 devices, 797 alerts and 29,756 telemetry readings. Reading counts quoted elsewhere rise from 29,512 (Figure 14) to 30,985 (Table 3) and 33,136 (Figure 8) because the simulator kept inserting between captures.

Table 8: Datasets used to test the implementation.

Dataset	Source	Used for
Seeded history	Synthetic readings generated by <code>src/seed.js</code>	Index benchmark, consumption analytics, backup and restore
Live stream	Physics-based simulator publishing over MQTT QoS 1	Ingestion, replication, failover and dashboard
Schema demonstration	One v2.4.1 and one v2.5.0 payload from <code>scripts/demo-schema.js</code>	Schema evolution (Figure 1)
Leak demonstration	Readings injected by <code>npm run demo:leak (source: demo_leak)</code>	Leak alert and dashboard banner
Failover probe	One probe document every 500 ms from <code>scripts/measure-failover.js</code>	Failover and quorum tests

Listing 2 shows an abridged v2.5.0 reading, the same document as in Figure 1. Stored documents also carry `device_id`, `device_type`, `location`, `alert`, `alert_reasons`, `ingested_at` and `source` (Figure 20).

Listing 2: Abridged `sensor_activations` document (firmware v2.5.0).

```

1 {
2   _id: ObjectId("6abfb372f9f8340c0d9e7d63"),
3   metadata: { firmware: "v2.5.0", hardware_rev: "ESP-WROOM-32U" },
4   timestamp: ISODate("2026-10-02T20:25:33.092Z"),
5   telemetry: {
6     water_tank: { ultrasonic_depth_pct: 59.5, volume_litres: 1190, distance_cm:
7       81 },
8     float_switches: { high_level_overflow: false, low_level_dry_run: false },
9     actuator_states: { inlet_valve: "OPEN", booster_pump: "ACTIVE" },
10    water_quality: { tds_ppm: 190, ph: 7.3 }    // field added in v2.5.0
11  }

```

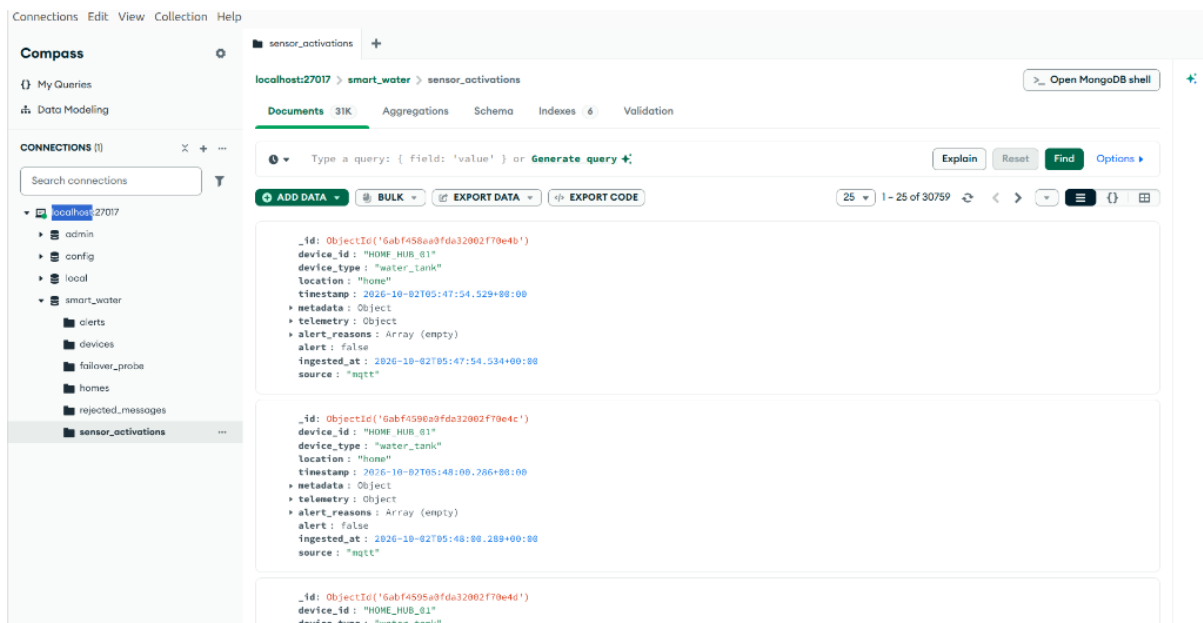


Figure 20: MongoDB Compass document explorer showing the BSON data structure.

Appendix B: Local Replica Set Deployment Commands

```
1 # Terminal 1: Node 1 (Primary candidate)
2 mongod --replSet rs0 --port 27017 --dbpath ./mongo-cluster/node1 --bind_ip
   localhost
3
4 # Terminal 2: Node 2 (Secondary)
5 mongod --replSet rs0 --port 27018 --dbpath ./mongo-cluster/node2 --bind_ip
   localhost
6
7 # Terminal 3: Node 3 (Secondary)
8 mongod --replSet rs0 --port 27019 --dbpath ./mongo-cluster/node3 --bind_ip
   localhost
9
10 # Initialize replica set once in mongosh
11 rs.initiate({
12   _id: "rs0",
13   members: [
14     { _id: 0, host: "localhost:27017", priority: 2 },
15     { _id: 1, host: "localhost:27018", priority: 1 },
16     { _id: 2, host: "localhost:27019", priority: 1 }
17   ]
18 });
```

Appendix C: Verification and Test Suite Status

The automated test suite was executed with `npm test (src/test.js)` and achieved **21 passed tests and 0 failures**:

- **Schema validation and DLQ:** rejects invalid timestamps and malformed payloads and routes them to the dead-letter queue.
- **Closed-loop hysteresis:** dual-threshold hysteresis prevents relay chattering on the inlet valve and booster pump.
- **Dry-run and overflow safety:** threshold rules raise alarms at or below 25% and at or above 85%.
- **Idempotent messaging:** the unique `(device_id, timestamp)` index deduplicates repeated MQTT QoS 1 frames.
- **CRUD:** verified across `homes`, `devices` and `sensor_activations` with API key authentication.
- **Aggregation pipelines:** validates `$dateTrunc` and `$setWindowFields` calculations for non-refill volume drops.
- **Leak engine:** verifies sensor-noise tolerance, overnight slow-leak detection and alert cooldown.
- **Depletion prediction:** validates drain-rate projections and graceful degradation on sparse data.
- **Cluster health:** confirms three-member connectivity, majority write concern and secondary-preferred read routing.